# Specification-Driven Software Reconstruction: From Independent Reproduction to Behavioral Discovery

Rafael Zamora-Resendiz
Lawrence Berkeley National Laboratory
rzamoraresendiz@lbl.gov

Preprint — October 2026

## Abstract

Generating a plausible implementation of a software component has become inexpensive, while establishing what that component is *required* to do has not. Conventional reproducibility addresses a different question: given fixed sources, it recovers a fixed artifact. It therefore offers no account of software whose source is itself generated, and no way to decide whether a regenerated implementation is the same software as the one it replaces.

We investigate whether the behavioral requirements of a system are sufficient to permit independent reconstruction of its implementation. We introduce a three-machine protocol that separates original production (M1), byte-preserving transfer (M2), and independent reconstruction from disclosed requirements alone (M3), together with a content-addressed identity computed over acceptance requirements while excluding generated implementation bytes. Repeated reconstruction then becomes an instrument: implementations admitted by the same requirements are sampled, probed differentially and against downstream consumers, and the resulting failures are converted into stronger requirements under explicit human authorization.

We report results from a system that reconstructs itself and from an external target. Three reconstructions of a 21-component self-hosting system qualified at one frozen identity across two producer families, including one whose kernel operations were executed entirely by a prior reconstruction; measured cross-family text similarity is 0.123. Six independent reconstructions of a configuration parser all satisfied its tests and all broke its consumer identically. On `jq`, a conforming reconstruction proved to be a lookup table, and runtime-generated behavioral laws subsequently closed 15 of 53 consistently divergent probes against one reopened. Preregistered predictions that failed are reported as such. One leg of the protocol is itself unmet: no result has been reproduced by an operator outside the authors' environment, so the self-hosting evidence demonstrates the protocol's components rather than a completed instance of it.

## 1 Introduction

Reproducing a computational artifact and reconstructing a software system are different problems. The first asks whether a byte sequence can be recovered: given the same sources, toolchain and environment, does the build produce the same output? Substantial infrastructure now answers that question well [25, 14, 12]. The second asks whether the *requirements* that a system satisfies are sufficient for someone else to build a system that satisfies them too. That question has received far less attention, because until recently producing a second independent implementation of anything was expensive enough that the question was academic.

It is no longer expensive. Large language models now produce syntactically valid, test-passing implementations of components at a cost low enough to repeat the exercise many times [7, 2, 22]. This changes what can be measured. If an implementation can be regenerated on demand, then the implementation is no longer the scarce object; the statement of what it must do is. And if generated implementations are going to be relied upon, some means is needed of saying what they were required to do, in a form that can be checked again later, by someone else, on another machine.

Three observations motivate the work.

*First, source code encodes decisions that its documentation and tests do not express.* This is an old observation about design intent [33] and about recovering design from artifacts that no longer explain themselves [9]. An implementation resolves questions that nobody wrote down: what happens on a duplicate key, whether a numeric-looking value is coerced, which inputs are errors. These resolutions become load-bearing for consumers whether or not they were ever specified.

*Second, independent implementations can satisfy the same tests and still disagree on consequential behavior.* The classical diversity literature established that independently developed programs fail in correlated ways [23, 15, 27]. We observe a sharper version: implementations generated from the same test suite can agree with each other, pass every test, and be jointly wrong about what a downstream consumer needs. Agreement among independent implementations is not evidence of correctness, and in our experiments it was actively misleading.

*Third, repeated independent reconstruction is now a fea-*

*sible experimental procedure.* A specification can be frozen, handed to several producers that do not see the original implementation, and the resulting implementations compared against one another, against held-out observations, and against real consumers. Where they diverge consequentially, the specification was silent. This turns underspecification from something discovered by accident in production into something that can be sampled deliberately in a laboratory.

The research question follows:

> *Can independent reconstruction serve as an experimental instrument for measuring and reducing behavioral underspecification?*

Our approach has two parts. The first is a protocol, the Three-Machine Test (§3), which separates the claim that an artifact transfers intact from the claim that its requirements suffice to obtain it. The second is a loop (§5) that uses repeated reconstruction, differential probing and consumer testing to produce failure witnesses, and converts witnesses into additional acceptance requirements under explicit human authorization. Both are implemented in an open system, *reticuli* [41], which serves here as the experimental platform rather than as the contribution.

**Contributions.** (i) A three-machine protocol that distinguishes artifact verification from implementation reconstruction, with a three-valued verdict in which unmeasured conditions can never accept (§3). We propose the protocol and report laboratory evidence for its components; its transfer leg has not been completed by an independent operator, and by the protocol's own rule that leaves our self-hosting verdict incomplete rather than accept. (ii) An implementation-independent, content-addressed identity defined over executable acceptance requirements and excluding generated implementation bytes, with the two conditions its committed byte set must satisfy (§4). (iii) A method for experimentally locating underspecification through repeated reconstruction, differential probing and consumer substitution (§5). (iv) Evidence from recursive self-reconstruction and from an external target that witnessed failures can be converted into stronger acceptance requirements, with measured effects where they exist and negative results where they do not (§7). (v) A framework for studying the cost and reliability of this process, with the economic questions stated as hypotheses requiring measurement rather than as findings (§8).

We state the support for each claim explicitly. A passing test suite establishes conformance to that suite. It does not establish semantic completeness, correctness, or that a producer worked without prior exposure to the target. Every self-hosting result reported here was obtained on hardware the authors control, and we mark it accordingly.

# 2 Background and Related Work

## 2.1 Reproducible computation and content addressing

Reproducible builds aim to make the mapping from sources to binaries deterministic, so that an independent rebuild yields bit-identical output [25]. Functional package managers realize this by making build inputs explicit and addressing the result by a hash over them [14, 12]. Bootstrappable builds reduce the trusted binary seed required to begin [32]. Content addressing itself, naming data by a hash over its bytes, is standard [30, 5].

All of this fixes the implementation and varies nothing. The guarantee sought is *identity of artifacts*. Our setting inverts the quantifiers: the requirements are fixed and the implementation is permitted, indeed expected, to vary. A reproducible build needs no notion of which alternative artifacts would have been acceptable, because exactly one is. Once many artifacts are acceptable, naming requires deciding which bytes may influence the name, which is the subject of §4.

Thompson's argument that a tool's capacity to reproduce itself says nothing about its trustworthiness [38] remains the relevant caution for any self-hosting result, and diverse double-compiling [39] is the closest prior method for attacking it. Our question is adjacent but distinct: not whether an executable corresponds to its source, but whether fresh source can be obtained from committed requirements and then perform the work of the tool that stated them. We eliminate neither the trusted runtime nor the producer.

## 2.2 Specifications and program synthesis

Executable specifications and contracts have a long history [31, 20]. Inductive program synthesis constructs programs from examples [18], and counterexample-guided inductive synthesis (CEGIS) alternates between candidate construction and counterexample generation [37, 1]; counterexample-guided abstraction refinement (CEGAR) applies the same shape to model checking [11]. Verified compilation relates an implementation to a formal semantics with machine-checked proof [26, 24], and regression verification establishes equivalence between program versions [17].

Our loop is structurally similar to CEGIS and deliberately weaker in what it concludes. CEGIS refines a candidate against a specification taken as ground truth. We refine the *specification* against evidence from candidates, consumers and independent implementations, and the refinement step is performed by a human who must authorize it. We prove nothing about semantic equivalence. The output of a successful round is a specification that excludes a witnessed class of unintended implementations, not a theorem.

## 2.3 Differential, property-based and metamorphic testing

Differential testing compares implementations of the same interface and treats disagreement as a bug signal [29], with com-

piler testing the canonical application [40]. Property-based testing [10] and metamorphic testing [8, 36] supply oracles where no reference output exists, addressing the oracle problem [4]. Dynamic invariant inference proposes properties that an existing implementation appears to maintain [16]. Mutation analysis measures the adequacy of a test suite by its ability to detect seeded faults [13, 21].

We use all of these as components. Two things are different. First, the implementations being compared are generated rather than independently developed by separate teams, which makes the comparison cheap and repeatable but introduces shared priors as a confound (§2.5). Second, the generated-input obligations are placed *inside the committed identity*, which forces a requirement that is otherwise optional: the inputs must be drawn from entropy at evaluation time. An obligation with fixed inputs is a finite test set, and Proposition 1 applies to it.

One difference bounds what a baseline comparison could show, and is worth stating directly. Differential testing against existing implementations can find behavioral divergence, and for the divergences in §7.4 it plausibly would have. It cannot find that section's first-generation failure, because that failure is a property of the *specification* rather than of any existing implementation: the degenerate member must be constructed before it can be observed, and constructing it is what generation does. We ran no such comparison and claim no efficiency advantage; the point is that the two methods have different reach.

Disagreement locates an investigation target. It does not say which implementation is right. In our external experiment the reference implementation was the sole permissive party in one divergence and hung on another, so "the reference is correct" is not available as a default.

### 2.4 Independent implementations and diversity

N-version programming proposed accepting any of several independently developed implementations [3]. Knight and Leveson showed experimentally that independently developed programs fail in correlated ways [23], and the theory of coincident failures followed [15, 27]. The practical obstacle was always cost.

That obstacle has changed, and the correlated-failure finding has not. It recurred in our experiments in the strongest available form: six reconstructions from two unrelated producer families, all conforming, all breaking a downstream consumer in the same way (§7.3). Cheap diversity does not purchase independence of judgment.

### 2.5 Generative software engineering

Benchmarks for code generation measure functional correctness against test suites [7, 2, 19]. Their adequacy has itself been questioned: strengthening the tests of an existing benchmark substantially lowers measured pass rates, indicating that the original suites admitted implementations that were not correct [28]. Agentic settings extend this to repository-scale tasks [22]. Security evaluations of generated code report systematic defect classes [34]. Memorization of training data is measurable and scales with model capacity [6], and benchmark contamination is a recognized threat to evaluation validity [35].

Both the opportunity and the risk in our method come from this literature. The opportunity is that independent reconstruction is now affordable enough to run as an experiment. The risk is twofold: a producer may reproduce a target it has memorized rather than derive it from the requirements, and two producers may share a prior and thus fail to be independent in the sense the method needs. We address the first by withholding the implementation and measuring artifact diversity, and the second by using two unrelated producer families and reporting cross-family results separately. Neither measure is conclusive, and §9 states what remains open.

### 2.6 What this work adds

Relative to reproducible builds: a notion of software identity that survives reimplementation, with the conditions under which it is sound. Relative to synthesis: a procedure that treats the specification, not the candidate, as the object under repair, with human authorization as a required step. Relative to differential and metamorphic testing: the placement of generated-input obligations inside a committed identity, and the use of reconstruction samples rather than existing implementations as the source of disagreement. Relative to N-version programming: a demonstration that the correlated-failure problem persists when versions are generated, and an instrument that detects it through consumers rather than through mutual comparison. Relative to code-generation evaluation: a method whose unit of measurement is the adequacy of the specification rather than the pass rate of a model.

## 3 The Three-Machine Test

A *claim* is a specification together with its sealed evidence: the executable acceptance obligations, the inputs they are pinned to, the recipe describing which outputs are generated and how, reproducibility metadata, and the generated implementation artifacts themselves. The protocol evaluates a claim on three machines.

**M1 — production.** The original environment. An implementation is produced, the acceptance obligations are executed against it, and the claim is sealed: its identity is computed and recorded in a manifest.

**M2 — transfer.** A byte-preserving copy of the claim is exported, imported into a different environment, and checked there. The identity is recomputed from the received bytes alone and compared with the manifest, and the acceptance obligations are re-executed.

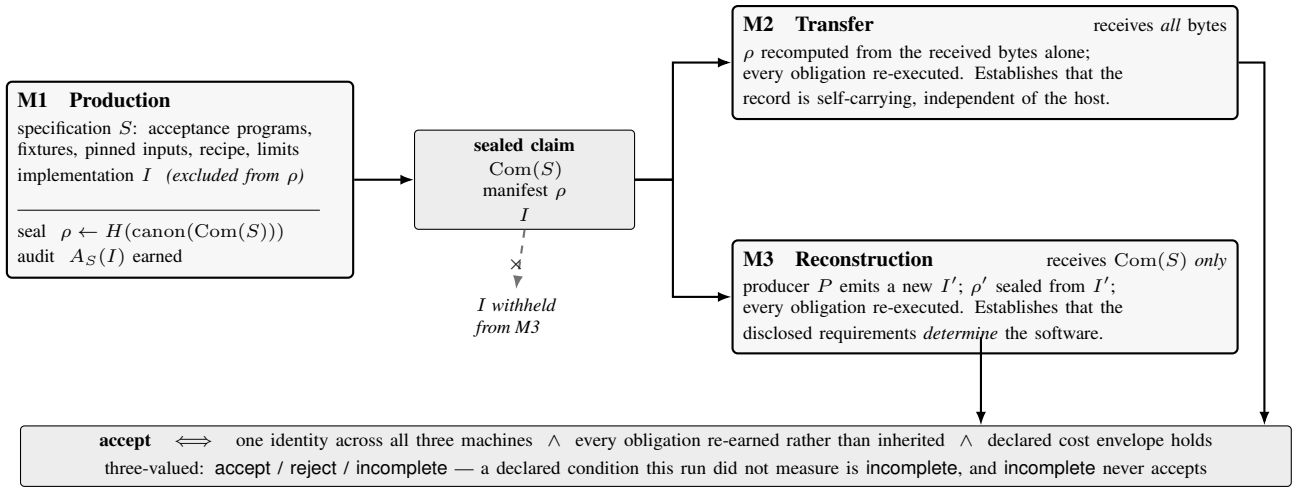



Figure 1: The Three-Machine Test. M2 and M3 answer different questions. M2 receives every byte and establishes that identity and verdicts survive transport. M3 receives the committed requirements only and establishes that they determine a conforming implementation. The protocol records information relevant to producer independence but cannot establish it from program bytes.

**M3 — independent reconstruction.** A workspace is materialized containing the committed material only: obligations, fixtures, pinned inputs and recipe. The original implementation is not present. A producer generates a new implementation, which is then sealed and judged.

The distinction between M2 and M3 is the point of the protocol. M2 tests whether a record is self-carrying: whether the evidence travels without the authoring host. This is the property reproducible-build infrastructure is designed to provide, and a failure of M2 is a defect in packaging or in the identity computation. M3 tests whether the disclosed requirements are sufficient to obtain a conforming implementation. A failure of M3 is a statement about the specification, not about the transport.

## 3.1 The verdict

A crosscheck accepts when one identity holds across all three machines, every obligation is re-earned rather than inherited from a record, and the declared cost envelope holds. The cost comparison uses the strongest unit both M1 and M3 recorded, in the order `usd` > `tokens` > `calls` > `seconds`; the default tolerance on the M3-to-M1 ratio is 2.0, and a claim or a verifier may set any value in $[1.5, 4.0)$. Where a claim declares a mutation floor, M3 must re-earn that too, with deterministic mutants drawn from the identity and a required kill rate.

The monetary unit needs a caveat that our own runs make concrete. Both producer families here ran on flat-rate subscriptions, so each recorded zero metered expenditure against a declared ceiling of $40. Zero measured spend and zero marginal cost are different states, and a protocol that conflates them lets a cost condition appear satisfied without constraining anything. By the three-valued rule below, an unmeasured declared condition should read incomplete rather than satisfied; a flat-rate run does not measure marginal monetary cost and should fall back to the strongest unit it did measure — tokens, calls or elapsed time. Our implementation does not currently make that distinction for the declared ceiling, and we report the resulting figures as unmeasured rather than as satisfied constraints (§C).

The verdict is three-valued: accept, reject, incomplete. A hard condition must hold to accept and rejects on failure. A hard condition that the claim declared but that this run did not measure is neither satisfied nor violated, and the verdict is incomplete. Incomplete never accepts. The rule exists because the common failure of verification tooling is to treat an unmeasured condition as a satisfied one, and the distinction between *missing evidence* and *rejection* is exactly what a reader of a verdict needs.

## 3.2 What the protocol does not establish

The protocol records which vendor and model produced M3 and marks same-vendor rebuilds as *independence unestablished*. It does not and cannot establish producer independence. Independence is a property of what a producer was exposed to, and no property of the emitted bytes decides it. We therefore treat the blindness of a reconstruction as a declared protocol with indirect support — the harness does not route the original implementation into the workspace, the resulting artifacts are measurably dissimilar (§7.1), and two unrelated producer families are used — and not as a verified fact. A successful reconstruction is an observation. It is not a proof

that the producer had no prior exposure to the target.

# 4 Implementation-Independent Identity

## 4.1 Construction

Let a specification $S$ be a finite map from paths to byte strings, each path carrying a declared role. Let an implementation $P$ be the material a producer supplies. An acceptance procedure evaluates an implementation under a declared execution environment $e$ and, where obligations draw generated inputs, a seed $\omega$:

$$A_S(P; e, \omega) \in \{0, 1\}. \tag{1}$$

The seed is not a formality. Section 5 requires it to be drawn fresh at each evaluation, so acceptance is a property of a *run*, not of an implementation alone. Two notions must therefore be separated. The verdict of a particular audit is $A_S(P; e, \omega)$. The set of implementations a specification admits *without qualification* is

$$\mathcal{I}(S) \;=\; \{\, P \;\mid\; A_S(P; e, \omega) = 1 \text{ for all } \omega \,\}, \tag{2}$$

which is not observable: no finite number of audits establishes membership. What an audit establishes is one draw. Where we report an implementation as admitted, we mean it passed the audits we ran, and the useful quantity for an unobserved future audit is a pass *probability* rather than a guarantee. We make no claim about that probability, having run too few draws per implementation to estimate it.

The identity is a hash over a declared subset of the specification's bytes,

$$\rho(S) \;=\; H\big(\mathrm{canon}\left(\{(p, H(S[p])) : p \in \Pi(S)\}\right)\big), \tag{3}$$

where $\Pi(S)$ is the *committed path set* and canon a canonical encoding. We write $\mathrm{Com}(S)$ for the committed content. Generated implementation bytes are excluded from $\Pi(S)$ by construction: in our implementation the identity computation skips every path whose declared class is *generated*. That exclusion is the mechanism by which $\rho$ names $\mathcal{I}(S)$ rather than any member of it.

## 4.2 Two conditions on the committed set

Choosing $\Pi(S)$ is the whole difficulty, and it is constrained from two sides. Define the *read set* $\mathrm{Rd}(A, S)$ as the set of specification paths whose contents can change the outcome of $A_S$ on some implementation, and $\mathrm{Sem}(S)$ as the paths carrying semantic content, as opposed to presentation.

**Condition 1 (Determination)** *For all $S_1, S_2$, all $P$, and every $e$ and $\omega$: $\rho(S_1) = \rho(S_2) \Rightarrow A_{S_1}(P; e, \omega) = A_{S_2}(P; e, \omega)$.*

Determination holds if and only if $\mathrm{Rd}(A, S) \subseteq \Pi(S)$ for every $S$, under three assumptions we state because they are load-bearing: that $A_S$ is deterministic given its declared execution environment and seed, that the environment is fixed across the comparison, and that $H$ is collision-resistant, so that equal identities imply equal committed content. The necessity direction is immediate: if some path outside $\Pi$ can influence an outcome, then two specifications differing only there share an identity and disagree on a verdict.

This failed in our system in an instructive form. Producer guidance — prose attached to a generated output, addressed to whoever writes it — had been removed from the committed set for reasons given below. The judging workspace, however, was still materialized from the *raw* specification, guidance included. An acceptance program that read its own guidance could therefore accept differently under one identity. Our documentation asserted this was impossible; a reproduction showed that as implemented it was not.

**Condition 2 (Admissibility)** *If $S_1$ and $S_2$ differ only on paths outside $\Pi(S)$, then $\rho(S_1) = \rho(S_2)$.*

This is invariance under excluded material, and it holds by construction for any $\Pi$. Its content is therefore entirely in the choice of $\Pi$: the smaller the committed set, the more verdict-irrelevant material the identity ignores. Determination bounds how small it may be, so the design goal is

$$\Pi(S) \;=\; \mathrm{Rd}(A, S), \tag{4}$$

committing every acceptance-relevant path and no other.

We state the condition this way because the stronger form is unobtainable, and an earlier version of this paper asserted it. One might want: if $S_1$ and $S_2$ admit the same implementations, then $\rho(S_1) = \rho(S_2)$. No choice of $\Pi$ delivers that. Both $\Pi$ and Rd are sets of *paths*, while verdict-irrelevant variation also occurs *within* a path. Adding a comment to a committed acceptance program changes no verdict and leaves the read set identical, yet changes the bytes at a committed path and so changes the identity. Delivering invariance there would require canonicalizing programs up to semantic equivalence, which is not available in general. We therefore claim path discipline, not semantic canonicalization.

The residue is not hypothetical, and it has cost us. A comment-only change to a committed obligation moved an identity, and under the strict reading of the no-intervening-change condition that invalidated a trial which had already qualified; it was regrown at the new identity (§7.1). That is this limitation in production.

Where the irrelevant structure is *known and finitely describable*, partial canonicalization recovers some of the gap, and both of our witnesses below are of that kind. They work because they canonicalize a specific construct in the preimage, not because they decide equivalence.

A violation of the condition does not produce a wrong verdict; it produces a wrong *name*. An implementation satisfies every obligation, receives a different identity, and is rejected as a different thing. We observed this twice. First with prose:

an independently written packager, conforming in every behavioral respect, phrased generated guidance differently and minted a different identity for an entire dependency chain. Second with order: once prose was removed, the residue was sequence. The step list was committed as a list, and two conforming packagers enumerated the same steps in different orders — one lexicographic, one following glob expansion — drifting 13 of 20 chain identities. The committed encoding now sorts steps canonically, making the list a set.

Both witnesses are instances of the same move: material was taken *out* of $\Pi$, or canonicalized within it, once it was shown to carry no verdict. Neither widened the committed set.

Three notions must be kept apart here, and conflating them is the error we made. A path may be *semantically relevant* — carrying real information about what is wanted — without being *acceptance-relevant*. Producer guidance is exactly such material: it tells an implementer what is intended and, by design, must never change a verdict. Writing $G(S)$ for guidance, the design constraint is $G(S) \subseteq \mathrm{Sem}(S) \setminus \mathrm{Rd}(A, S)$, and (4) then excludes guidance from the identity automatically. An earlier formulation of ours placed $\mathrm{Sem}(S)$ as an upper bound on $\Pi(S)$ and described the gap between $\mathrm{Rd}$ and $\mathrm{Sem}$ as designer freedom. That was wrong: admissibility as stated admits no such gap, because material that cannot change a verdict must not be committed whether or not it is meaningful.

The practical consequence survives the correction, and is the reason the obvious repair to a determination failure is the wrong one. When a byte must be excluded for admissibility yet the acceptance program can read it, widening $\Pi$ is unavailable. The only sound repair is to shrink $\mathrm{Rd}$ — to make the byte *unreadable* by separating the context in which an implementation is constructed from the context in which it is judged. In our system the judging workspace is now materialized from the committed content itself, so no acceptance program can read what the identity does not cover.

### 4.3 Composition

Specifications depend on other specifications. Let $C$ declare edges $(d, o, i)$ meaning that component $d$ supplies output $o$ to $C$'s input $i$, and let $C$ ship bytes $\beta_C(i)$ at that input. The composed verdict must be

$$A^*(C) \;=\; A_C \;\wedge \bigwedge_{(d,o,i)} A_{D_d}\big(\beta_C(i) \text{ as } o\big). \qquad (5)$$

Each component's own acceptance procedure is re-executed on the bytes the dependent actually ships. A recorded verdict from the component's history is not evidence about the dependent's bytes, because the component's identity names a class and the dependent links one member of it.

Equation (5) was violated in three places in our implementation, each exploitable. A dependency attribution recorded at seal time and never re-checked allowed a claim to assert an origin its bytes did not have, and the forged claim passed identity verification, shallow acceptance, composed acceptance and the dependency view in every implementation we tested, including the reference one. A signing ceremony folded every declared component into the signed identity while re-earning only $A_C$, so a claim whose own composed audit rejects it could be signed. And the user-facing audit verb resolved to composed acceptance by behavior alone, with no obligation requiring it, so a reconstruction was free to wire the verb to the shallow primitive and pass every test.

The three share a structure. In each case the obligations exercised one side of a two-sided contract: code edges were re-earned but data edges were not; the signing ceremony's refusal behavior was tested but its composed refusal was not; the verb's denial behavior was pinned but its permission behavior was not. This generalizes into an audit procedure. To look for omissions in a specification, enumerate the symmetries of its contracts and check whether both halves are exercised.

### 4.4 Test-based and behavioral equivalence

Equation (2) defines equivalence relative to an acceptance procedure, which is weaker than behavioral equivalence and must not be confused with it. $\mathcal{I}(S)$ can contain mutually incompatible implementations whenever $A_S$ omits a requirement that some consumer depends on. Section 7.3 exhibits exactly this: six implementations, all in $\mathcal{I}(S)$, none usable by the consumer. An identity over $S$ therefore names *a declared experiment*, not a semantic property of programs. Two specifications that test the same behavior by different means receive different identities; identity equality asserts the same question, not the same answer.

### 4.5 Specification transitions

Strengthening or otherwise changing the obligations produces a new identity. Our implementation treats this as a deliberate, recorded, human-authorized act: the lineage of revisions is retained, each transition names the failure that motivated it, and no automated step may perform one. The practical significance for the experiments below is that adopting a new obligation invalidates prior qualifying results at the old identity, which is why the counts in §7.1 reset as often as they do.

## 5 Reconstruction as Specification Discovery

Repeated independent reconstruction samples $\mathcal{I}(S)$. If the sampled implementations disagree consequentially, or if any of them breaks a consumer that the original satisfied, then $A_S$ omits a requirement. The loop in Figure 2 makes this a procedure:

1. Freeze a specification and its acceptance boundary.
2. Generate independent candidate implementations (M3, repeated).
3. Confirm that each is admitted by the frozen boundary.

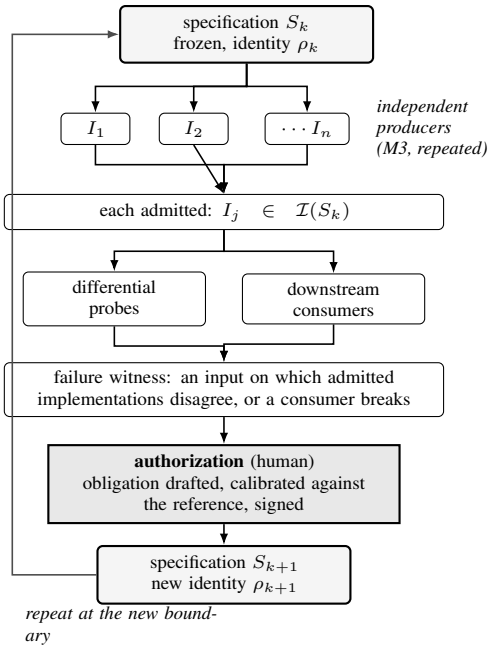



Figure 2: The specification refinement loop. Independent reconstructions sample $\mathcal{I}(S_k)$; differential probes and downstream consumers produce failure witnesses; an authorized refinement yields a new specification version $S_{k+1}$ with a new identity. The frozen specification is never mutated in place.

4. Probe the candidates against held-out observations, downstream consumers, and generated behavioral probes.
5. Identify consequential divergence or consumer failure, and retain the witnessing input.
6. Convert justified witnesses into additional acceptance obligations, calibrated against the reference before adoption.
7. Seal the revised specification under a new identity, and repeat.

### 5.1 Measured quantities

The loop defines several measurable quantities: the acceptance rate of independent reconstructions at a boundary; cross-implementation disagreement, measured per probe; consumer compatibility, measured as the fraction of admitted implementations a consumer can use; reconstruction cost in the units the producer reports; the distribution of failure classes; and the change in disagreement following a repair. The last of these is the one most easily measured wrongly, and §7.4 reports an instance in which we did so.

### 5.2 Two limits of the instrument

**Agreement is not correctness.** Producers may share an interpretation, in which case the sampled implementations converge on a reading that is uniformly wrong. This is the correlated-failure result [23] reappearing with generated versions, and it bounds what mutual comparison can find. Only an external oracle — a consumer, a reference implementation, a human judgment — can detect a shared error. Our strongest instance is in §7.3.

**Finite test sets admit memorization.**

**Proposition 1 (Extensional insufficiency)** *Let $S$ have acceptance obligations consisting of a finite set of input-output observations $\{(x_j, y_j)\}_{j \le n}$ compared exactly. Then $\mathcal{I}(S)$ contains an implementation of size $O(n)$ that stores the observations, answers $y_j$ on $x_j$, and is unconstrained on every other input.*

The proposition is trivial as mathematics. It is reported because the degenerate member is not hypothetical; §7.4 records a reconstruction that realized it, documented what it had done, and passed. The repair is to add obligations whose inputs do not exist when the implementation is written: computed relations, in which the checker derives the expected result independently for each generated input; metamorphic relations between executions, which need no oracle [8]; and refusal obligations, in which generated ill-typed inputs must raise, guarded so that refusing everything cannot satisfy them.

These do not categorically exclude table-based strategies, and we do not claim they do. They change the economics. Memorizing the original finite corpus no longer suffices, because the inputs that decide the verdict do not exist when the implementation is written; a table would have to cover the generator's reachable input space, which for the obligations we use is far larger than the corpus and in the limit unbounded. The degenerate member is made expensive rather than impossible, and an implementation that memorized the *generator* would still defeat them.

Two consequences follow, both learned the hard way. The entropy must be drawn fresh at each evaluation; a fixed seed converts the obligations into a larger finite test set, and Proposition 1 applies again. And each obligation must be calibrated against the reference before it is committed, because an obligation that misstates the reference's behavior refuses conforming implementations (§7.4).

## 6 Experimental Design

We organize the evidence around five questions (Table 1).

**RQ1** *Independent reconstruction.* Can distinct implementations satisfy the same sealed specification without reusing the original implementation?

**RQ2** *Recursive self-hosting.* Can a reconstructed implementation perform the verification and reconstruction operations required to produce subsequent implementations?

**RQ3** *Underspecification.* Do independent reconstructions and external consumers expose behavioral obligations missing from the acceptance criteria?

Table 1: Experimental design. No experiment was run for this manuscript; all were conducted during the development of the platform, and the preregistration applies to the `jq` arm only.

| | target | producers | outcome measure |
|---|---|---|---|
| **RQ1** independent reconstruction | 21-component self-hosting system | codex, claude | acceptance at frozen identity; artifact similarity |
| **RQ2** recursive self-hosting | the same, generationally | codex authored, claude executing | identity preserved with kernel acts performed by a reconstruction |
| **RQ3** underspecification | configuration parser with a consumer | 3 codex models, 3 claude models | admitted implementations that break the consumer |
| **RQ4** specification repair | self-hosting system; `jq` | both families | new failures per generation; paired per-probe divergence |
| **RQ5** generalization | `jq` 1.8.1 (external) | codex, claude | divergence recall against an independent human-divergence surface |

**RQ4** *Specification repair.* Can witnessed failures be converted into requirements that reduce measurable consequential divergence?

**RQ5** *Generalization.* Does the method reveal useful underspecification in independently developed external software?

**Producers and environments.** Two unrelated producer families were used throughout: a *codex* family (models recorded as `gpt-6-sol`, with `gpt-6-luna` and `gpt-6-astra` in the substitution experiment) and a *claude* family (`sonnet`, with `opus` and `haiku` in the substitution experiment). Reconstruction ran on a single `arm64` Darwin host with CPython 3.14, acceptance obligations executed under a sandbox profile that denies network access and writes outside the workspace. Across 26 recorded whole-system reconstructions the median wall-clock was 40 minutes for the codex family and 186 minutes for the claude family, over 21 components, with 0–1 and 0–4 per-component retries respectively.

**Preregistration.** The `jq` arm was preregistered: its predictions were recorded and signed before the first reconstruction workspace was opened. The self-hosting arm was not preregistered. It was conducted as development, and its results are reported as such. We note this asymmetry because it limits what the self-hosting numbers can support: they are measurements of a system under repeated repair, not the outcome of a design fixed in advance.

**Contamination.** The external target is widely represented in producer training data. Its agreement results are therefore uninformative and we use only its divergences. The self-hosting target is the authors' own unpublished system at the time of the experiments, which reduces but does not eliminate the concern, since blindness remains declared rather than enforced (§3).

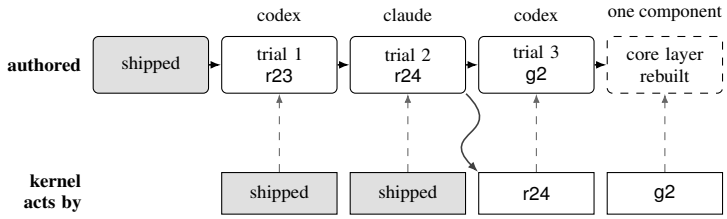



Figure 3: Recursive self-hosting lineage at frozen identity `900b6594`. The lower row records which implementation executed each generation's kernel operations.

Table 2: Reconstructions at the final frozen identity `900b6594`, each sealed as a claim whose own gate re-earns the verdict. No row has been reproduced by an external operator.

| trial | sealed claim | producer | executed by |
|---|---|---|---|
| 1 | `61d77f1f` | codex | shipped |
| 2 | `29b185a8` | claude | shipped |
| 3 | `2568e7d4` | codex | trial 2 |

# 7 Results

## 7.1 Independent and recursive self-reconstruction

The system was decomposed into 21 components and reconstructed in dependency order, each component's producer receiving already-reconstructed dependencies as context and never the original component. Assembled implementations were then judged against the unchanged whole-system identity with all 37 generated modules substituted.

At the final frozen identity `900b6594`, three reconstructions qualified (Table 2): one from each producer family, and a third in which every component was authored blind by one family while every kernel operation — sealing, sandboxing, rebuilding, computing identities — was executed by the *other* family's reconstructed code, with no shipped code in the loop. That third result is the one that answers **RQ2**, because it trans-

fers the operational role rather than only the implementation: the tool performing the work is itself a reconstruction. Its own recursive step then exercised the operational interface directly: using the reconstruction's own command-line tool, a component claim was exported blind, imported into a fresh workspace, rebuilt through a producer, and found to carry the identity of the original, with the parent verifying the child. We state the scope of that step precisely, because it is easy to overstate. It rebuilds *one component* — the core layer, identity `4c2abcbf` — and not a further whole-system generation. What it establishes is that a reconstruction can perform the export, rebuild and verification operations the method requires, not that a fourth complete system was grown.

Each qualifying reconstruction is sealed as a claim whose own acceptance procedure re-executes the conditions, so the verdict is re-earned rather than asserted. A claim cannot seal unless its obligations pass, and two non-qualifying trials in this series have no sealed manifest, which we verified directly. The construction also refused a seal over a boundary whose own obligations named a path outside the claim, which was then fixed.

**Implementation diversity.** The measured population is eight complete implementations: *seven independent reconstructions* (three codex, four claude) together with the shipped implementation as a reference point. Similarity figures between reconstructions are computed over the seven; the shipped tree enters only where it is named.

Among the seven, mean pairwise text similarity is $0.123$ across producer families, $0.317$ within the codex family and $0.201$ within the claude family. Of the 37 modules present in every implementation, 32 take a distinct syntactic shape in every reconstruction; the five that converge are package initialization files and the top-level entry point, which contain almost nothing to write. Reconstruction size is bimodal by family: 4,384 to 4,842 lines for codex and 8,381 to 8,638 for claude. The shipped implementation, scored against the seven, is the least typical member of its own admitted class at similarity $0.074$. Measured with comments, docstrings and layout normalized away, the same figures are $0.198$, $0.349$ and $0.290$; we report both channels because the raw figure credits prose differences as implementation diversity and the normalized figure does not.

The identity is therefore not accidentally stable. It survived three complete replacements of the bytes it names, with the replacements measurably dissimilar to one another and to the original.

These are textual measures, and textual dissimilarity is weaker evidence than it may appear. It establishes that the reconstructions are not copies, which is what bears on memorization. It does not establish that they embody different algorithms, and we make no such claim: two implementations may share an approach and still share little text. Functional and structural diversity measures would settle this, and we have not applied them.

Table 3: Reconciliation of the reconstruction counts used in this paper. The populations are nested subsets of one series, not independent experiments.

| $n$ | population |
|---|---|
| 26 | whole-system reconstructions with a recorded producer ledger, across all boundary versions. The basis for the timing and retry figures of §6. |
| 8 | complete reconstructions retained as artifacts and used for the diversity measurements, drawn from several boundaries. Includes the shipped implementation as a reference point. |
| 7 | the same set excluding the shipped implementation: the reconstructions only, used for the per-component agreement gradient in the appendix. |
| 3 | reconstructions qualifying at the final frozen identity `900b6594` under every condition of §3 except the external leg. The basis for Table 2. |

**The cost of the result.** The final count of three was reached only after the count was reset to zero five times by adopted boundary changes, and once by a ruling that a trial whose authoring predated a comment-only identity change could not count under a strict reading of the no-intervening-change condition. One reset surrendered an already qualifying trial in order to adopt a requirement the authors judged more valuable than the count. A reader who learns only the final figure has been misled about the method's cost: the three qualifying reconstructions are the end of a sequence of approximately two dozen, most of which failed, and each failure that was adopted as a repair invalidated the results before it.

## 7.2 Specification repair during self-hosting

Failures were classified before any repair was drafted. Table 4 gives representative entries from the 19 adopted obligations. The recurring classes were: workspace and filesystem assumptions (pattern expansion, empty selections, flat versus nested staging); sandbox policy, where a reconstruction's own confinement profile was simultaneously strict enough to pass its tests and too strict to run the system's real acceptance gate; record and format round-trips between components written at different layers; composed verdicts that did not re-earn their parts; and resource growth.

Three features of this record bear on **RQ4**.

First, the *sequence of newly discovered failures per generation declined* across the codex arc, counted by the project at the time as 3, 3, 3, 2, 1, 0 for successive reconstructions. We report these as the authors' classification rather than as a mechanically derived series; the underlying per-revision records exist, but the counts were not emitted by a script.

Second, several obligations have direct post-adoption evidence that they steered subsequent reconstructions, noted in Table 4. The clearest is the sandbox obligation: the reconstruction immediately following its adoption passed both the

Table 4: Representative adopted obligations, the failure that selected each, and the evidence since adoption. Nineteen obligations are recorded in total; the project's revision lineage contains 34 recorded transitions.

| obligation adopted | failure that selected it | evidence since adoption |
|---|---|---|
| Pattern expansion applies to pinned inputs, not only generated outputs | A reconstruction expanded globs in one field and read the other literally; refused at 21 s | No recurrence across the following six reconstructions |
| The reconstruction workspace recipe is TOML | A reconstruction wrote canonical JSON into the recipe file, self-incompatible with its own format | No recurrence across the following six reconstructions |
| A component record round-trips between packaging and deep audit | A reconstruction sealed records as `{component, root}` while its own deep audit read an `input` key | Steering confirmed 4/4: the packaging layer passed first-attempt in the next four reconstructions |
| Components resolve by identity, never by physical nesting | Two independently written walkers in different families made the same assumption; 2 of 4 incidence cross-family | The transitivity fixture now stages components flat mid-battery |
| The sandbox must permit a real gate (user lookup) | A reconstruction passed identity but failed as *auditor*: its deny-default sandbox refused a user lookup the repository gate needs | Steering confirmed 1/1: the next reconstruction passed both the run layer and the repository audit, becoming the first qualifier in its family |
| A pattern may select the empty set | A reconstruction raised on a glob matching no file, where the chain's uniform early-layer patterns legitimately select nothing | The next reconstruction took the empty selections without complaint |
| A declared data dependency re-earns its content match | A forged attribution passed every verification verb in every reconstruction, the shipped implementation included | Refusal now names the attribution; the shipped implementation required a fix |
| The chain build is incremental (doubling cost bounded) | A reconstruction walked ancestor links once per reaching path: measured doubling ratio $\approx$95 against 2.1–3.5 for conforming implementations, with a $2^K$ model fitting the interpolated point to 1% | First obligation adopted on a time axis |

component gate and the whole-repository audit, becoming the first qualifying reconstruction from the claude family after seven consecutive refusals.

Third, and in the other direction, the second producer family's arc shows what repair does *not* buy. Seven consecutive reconstructions from that family were each refused by exactly one defect, each defect deeper in the system than the last, with three adopted obligations in between. Repair moved the failure rather than eliminating it, until the eighth reconstruction qualified. We cannot distinguish "the obligations were progressively completed" from "the family's failures were enumerable and we exhausted them" on this evidence.

**Obligations not adopted.** Three candidate obligations were drafted and rejected after measurement contradicted the diagnosis they encoded: one attributed a failure to a deduplication key, which measurement falsified; one assumed silent truncation where the implementation in fact raised; one isolated a mechanism that could not be encoded as a fixture and was accepted as already caught elsewhere. We record these because the discipline they represent is load-bearing: an authorization step authorizes an intent, and an intent resting on a wrong diagnosis produces an obligation that refuses conforming implementations.

**Resource obligations.** Functional obligations accept implementations with arbitrarily poor asymptotics, and absolute time bounds cannot be committed to a portable identity because they describe a machine rather than a program. Growth ratios can. For an operation parameterized by scale $K$, measured twice on the same host in the same run,

$$t(2K)/t(K) \;\leq\; c, \tag{6}$$

the host's constant factor divides out and what remains constrains the algorithm. A slow linear implementation on slow hardware passes; a fast exponential one fails everywhere. This form caught the traversal defect in Table 4 and a corresponding space defect, in which a copy procedure duplicated shared ancestors once per reaching path and a dependency store grew exponentially in chain depth, reaching approximately 23 GB for a 20-layer chain in ordinary use. The declared envelope bounds doubling on the recursion-bearing axes at $\times 8$ against a measured conforming band of 1.1–3.5. The limitation is that a ratio constrains the exponent only at the scales measured.

## 7.3 Consumer substitution

This experiment is the cleanest answer to **RQ3** and the sharpest illustration of the gap between test-based and behavioral equivalence.

A configuration parser $D$ was sealed with a deliberately partial boundary $C_0$ that pins word-valued, distinct-key assignments and the empty document, leaving numeric coercion and duplicate-key meaning unspecified. A consumer $P$ was sealed separately; it parses a document with a repeated `port` key and doubles the resulting value, expecting 160. $P$'s recipe makes the parser a generated file, so $P$'s identity commits to the contract and never to any parser's bytes, and a candidate parser is judged by auditing $P$ with that candidate substituted.

Six blind reconstructions of $D$ from $C_0$ alone were produced across two families and six models. All six were admitted by $C_0$. All six had distinct build digests. **None kept the consumer working.**

The failure was not six independent mistakes. Every reconstruction chose last-wins for duplicate keys, which the consumer needed, and kept values as strings, which broke it: the consumer's `cfg["port"] * 2` evaluated to `"8080"` by string repetition. The boundary was silent about coercion and both families' priors filled the silence the same way, on the side the consumer could not use.

The consequence for the method is stronger than the consequence for the parser. Six independent implementations that agree with one another all broke the consumer identically, so no amount of cross-checking reconstructions against each other would have surfaced the defect. Only the consumer did. This is the correlated-failure result [23] in a setting where diversity is cheap, and it bounds what differential comparison between reconstructions can establish.

Adding the two obligations the consumer relied upon — numeric values coerce, duplicate keys are last-wins — produced a tightened boundary $C_1$. Re-running the same six producers against $C_1$: all six were admitted and all six kept the consumer working. The difference is two pinned lines. One reconstruction needed a second attempt.

## 7.4 External software: `jq`

The external arm applies the method to software the authors did not write, where independently developed implementations already exist and can serve as a divergence surface. The target is `jq` 1.8.1, with `gojq` 0.12.19 and `jaq` 3.1.1 as oracles. The oracles adjudicate and are never used as acceptance criteria.

**The premise, measured without model spend.** Running the official `jq` corpus of 522 parseable cases against all three implementations, all three return the same verdict on 373 and differ on 149, a disagreement rate of **28.5%**. Of the 373 agreements, 332 are cases all three pass and 41 are cases all three fail identically. Per-implementation pass counts on `jq`'s own corpus are 479, 408 and 337. An officially maintained test corpus is therefore measurably insufficient to pin the behavior of its own language, as judged by independent human reimplementations. Approximately 43 cases fail even for the reference implementation under a plain runner and are excluded as harness artifacts. The initial specification $S_0$ is the 465-case evaluation-core slice.

**Generation 1: the degenerate member.** The first blind reconstruction converged on the first attempt, passed 465 of 465, and verified clean: a formal member of $\mathcal{I}(S_0)$. On 1,395 seeded probes drawn from outside the corpus it agreed with the reference on 587 and raised an error on the remaining 808. The reason is in the implementation's own header, which states that the supplied specification is an extensional set of observations and that the artifact is not a general interpreter. It is a 465-entry lookup table with careful input normalization. Proposition 1 is not a hypothetical: given a finite corpus and nothing else, a competent producer built the cheapest member of the admitted class and documented that it had done so. On the same probe set the human implementations split from the reference on 296 of 1,395, which is live divergence territory outside the corpus.

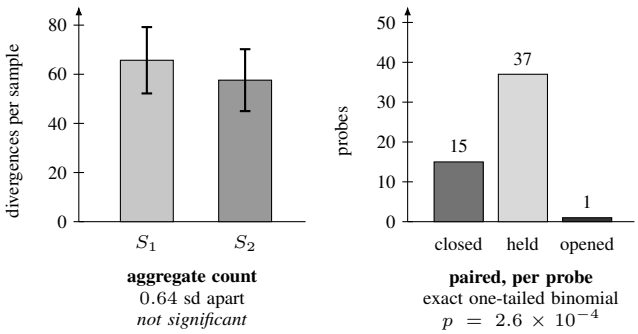
**Repairs.** Two revisions followed, each signed. $S_1$ added 16 metamorphic and computed laws over inputs seeded from entropy at evaluation time, validated on three fresh seeds against the reference. $S_2$ added 28 error-condition laws requiring that generated ill-typed inputs raise, each condition first verified unanimous across all three implementations, validated on five fresh seeds. Three further candidate laws were *rejected* by that validation before adoption, because the reference's refusals proved value-dependent rather than type-dependent: it reverses the number 0 to the empty list but raises on 5, since its definition routes through a length that is zero in the first case. Had those three been committed as drafted they would have refused conforming implementations.

**Preregistered predictions.** Of the predictions recorded before the first workspace opened, one was supported, one falsified, one returned nothing.

*Concentration (supported).* Pooled divergence lies on human-split probes 55.4% of the time against a 24.4% base rate, an enrichment of $2.30\times$. In the sharpest form, of the 19 probes where reconstructions from *both* families diverge from the reference, 17 (89%) are probes where a human reimplementation also disagrees. Independent machine priors and independent human teams locate the same silence.

*Recall (falsified).* The prediction was that sampled reconstruction would surface at least 30% of the human-divergence surface. Measured: **17.6%** (60 of 343 probes). Three codex samples reached 15.0%, and adding an entire second producer family moved this to 17.6%. The refined finding is more useful than the original guess: the silence that two frontier priors can jointly see is substantially narrower than the silence that demonstrably exists. Sampled reconstruction finds a concentrated and high-value fraction of the divergence surface, not all of it.

*Novel ambiguity (none found).* Under a strict criterion — a both-succeed value divergence, off the human-divergence surface, flagged by at least two independent samples — the measured count is **zero**. The off-surface multi-sample set consists entirely of cases where reconstructions succeed and the reference and both human implementations raise, which is a defect class rather than novel ambiguity. With contamination




Same samples, same probes, two instruments. The aggregate discards the pairing between a probe before and after the repair, and between-sample variance then exceeds the effect — a preregistered control at the *unchanged* specification returned the same aggregate, which is how the first, withdrawn version of this result was caught. Closures are traceable to the obligations responsible (`to_entries` $1.00 \to 0.00$, `indices` $0.67 \to 0.00$). Codex family only; the claude family shows no contraction (§7.4).

Figure 4: Behavioral divergence before and after repair, measured two ways on the same samples and probes.

as heavy as this target's, the priors reproduce documented semantics well and fail where the reference is silent about errors.

**Did the repair reduce divergence?** This question produced the arm's most transferable methodological result, and we report the error along with the correction.

An initial reading compared aggregate divergence counts per sample and appeared to show a 25% reduction. A preregistered control at the *unchanged* specification returned the same aggregate: pooled by boundary, $65.7 \pm 13.5$ before against $57.6 \pm 12.6$ after, a difference of 0.64 standard deviations. Between-sample variance exceeded the effect, and the claim was withdrawn within the hour.

Re-measured as a paired per-probe comparison — the same probe before and after, where a probe counts as consistently divergent if it diverges in at least two thirds of samples — the effect is present. At $n = 7$ and $n = 5$ samples: **15 probes closed, 37 held, 1 opened**, exact one-tailed binomial $p = 2.6 \times 10^{-4}$. The effect strengthened as samples were added, from 10:1 and $p = 5.9 \times 10^{-3}$ at the smaller sample. Closures are traceable to the obligations responsible, with `to_entries` moving from a divergence rate of 1.00 to 0.00 and `indices` from 0.67 to 0.00, both named explicitly in the error laws. Figure 4 shows both measurements on the same data.

The aggregate discards the pairing between a probe before and after the repair, and the pairing is where the signal is. Same samples, same probes, two instruments, opposite conclusions.

**A negative cross-family result.** The contraction above is measured on the codex family. On the claude family it does not replicate: paired at $n = 1$ per boundary, 12 probes closed, 22 held and 12 opened, a sign test of $p = 0.58$ and no contraction at all.

The class decomposition explains this, and the explanation is measurable. Divergence falls into classes, and the families have different profiles: excess permissiveness accounts for 54% of the codex family's divergence and 18% of the claude family's. The $S_2$ repair was 28 error laws, aimed squarely at permissiveness, and permissiveness fell in both families. But a repair that addresses a class constituting half of one family's divergence and a fifth of another's will move one aggregate and not the other. The repair did what it was designed to do; the measurement of whether a specification improved is not answerable from one family's aggregate.

We report this as a negative result and not as a resolved one. The claude leg is $n = 1$ and carries no statistical weight on its own. The account is falsifiable and the test is specified in advance: a pending revision targets the value-divergence class, and the account predicts that a fresh claude sample's value-divergence count will fall while its permissiveness count stays flat.

**Byproducts.** The probe batteries incidentally surfaced three undocumented divergences in the ecosystem, each reproducible with a single command: the reference returns an empty list where both reimplementations raise; one reimplementation fails to terminate on a generated range over a string; and the reference itself fails to terminate on a large string repetition, apparently building the result eagerly. Two of the three are in the reference implementation. These are byproducts of the method rather than contributions of this paper.

**An open decision.** Of the programs that diverge consistently at both boundaries, 19 reduce to four classes. Six differ only in error message text, which the project's precedent leaves unpinned. Nine have a single reimplementation as the outlier and admit a mechanical consensus resolution. Three constitute a genuine election: indexing an object by a string key where the reference returns null and *both* reimplementations raise. Pinning the reference's behavior requires members to replicate a permissive edge that two independent implementations treat as an error; pinning the majority produces a specification that refuses the artifact it was recovered from. This decision is

unresolved at the time of writing.

### 7.5 Consolidated findings

Table 5 separates what the evidence demonstrates, what holds only within our environment, what came out negative, and what remains open.

## 8 Discussion

### 8.1 Specification as a persistent asset

If a specification is sufficient to obtain a conforming implementation, then the specification and not the implementation is the durable object. This reframes a familiar problem. Software knowledge is ordinarily lost when an implementation becomes unmaintainable, because the requirements were never separable from the code that satisfied them. An executable specification with a stable identity is separable by construction, and our self-hosting result is evidence that the separation can hold through complete replacement of the implementation.

The claim must be bounded carefully. Specification identity is not semantic identity. An identity over $S$ names a declared experiment: two specifications that ask the same behavioral question by different means receive different identities, and identity equality asserts that the same question is being asked, not that the answer is correct. Section 7.3 is the cautionary case, in which the preserved specification was faithfully satisfied and nonetheless insufficient for its only consumer.

### 8.2 Applications

Several applications follow, each requiring technical work this paper does not supply.

*Legacy reconstruction.* Where a system must be replaced and its behavior is known only through the system itself, the method offers a procedure: recover a specification from observed behavior, then reconstruct against it and use divergence to find what the recovery missed. The `jq` arm is a small instance. The required advance is specification recovery from execution traces at scale, for which dynamic invariant inference [16] is the natural input.

*Compatibility and regression discovery.* Independent reconstruction is a source of inputs on which conforming implementations disagree, and such inputs are exactly the regression tests a suite is missing. The required advance is automatic triage: our divergence classes were assigned by hand.

*Vendor independence.* A consumer holding a specification with a credible reconstruction path has an exit option from any particular implementation. The required advance is a consumer-side acceptance boundary strong enough that substitution is safe, which §7.3 shows is a distinct and stronger property than component-level conformance.

*Assurance for generated code.* The method converts "the model wrote something that passes" into "the requirements are recorded, re-earnable, and known to admit at least these implementations." The required advance is cheaper obligations, since the ones here were written by hand.

### 8.3 Economic implications

We state these as hypotheses, because no cost study supports them. Our repository records a per-reconstruction envelope and some per-run figures, and compares nothing against an alternative method.

If generating an implementation becomes inexpensive while maintaining a reliable specification does not, then economic value should shift toward whoever holds the specification and the authority to judge conformance against it. Four effects would be measurable: migration cost when replacing an implementation; dependence on a particular vendor; time to discover an undocumented requirement; and reuse of acceptance criteria across implementation generations. Plausible business models follow — specification auditing, reconstruction infrastructure, continuous behavioral verification — but a model is not a measurement, and the determining quantity is the human adjudication cost per replacement, which we have not measured.

We note one consideration that cuts against naive optimism. The scarce asset in this scheme is a maintained acceptance contract, and whoever governs it holds the power the implementations lose. Consolidating specifications moves authority to the specification steward, which is a different risk from vendor lock-in rather than an absence of one.

### 8.4 Iterated reconstruction against strengthening specifications

The loop of §5 can in principle be iterated: reconstruct, find divergence, strengthen, reconstruct again. Our results are consistent with this accumulating knowledge while implementation strategies vary freely, and the self-hosting arc is the longest instance we have, at roughly two dozen reconstructions and nineteen adopted obligations.

It should not be read as autonomous self-improvement. Every adopted obligation in this work required a human authorization step, the diagnosis behind three candidate obligations was wrong and was caught only by measurement, and a producer improving its own acceptance boundary has no external check. The arrangement that produced our results keeps authorization outside the loop deliberately.

### 8.5 Substitutability, markets and intellectual property

If implementations become substitutable under a specification, two distinctions become practically important. The first is between *technical reconstructability* and the *legal right* to reproduce or distribute: that a conforming implementation can be obtained from a specification says nothing about whether doing so is permitted, and the specification may itself be derived from a protected artifact. The second is between behav-

Table 5: Findings by level of support. No row has been independently reproduced by an operator outside the authors' environment.

| | finding | support |
|---|---|---|
| **Demonstrated** | | |
| **RQ1** | Independent reconstructions satisfy a frozen specification and are measurably dissimilar: cross-family similarity 0.123, 32 of 37 modules distinct in every reconstruction. | Eight complete reconstructions measured; three qualifying at one identity; instrument reproduces a prior reading to $\pm 0.003$. |
| **RQ3** | Test-based admission does not imply substitutability: 6/6 reconstructions admitted, 0/6 usable by the consumer, failing identically. | Six reconstructions, two families, six models; repair verified by re-running the same producers (6/6 usable). |
| **RQ3** | A finite test corpus admits memorization, and a producer realizes it: 465/465 conformance by a 465-entry lookup table. | Implementation source preserved; 587 agreements and 808 refusals on 1,395 novel probes. |
| **RQ5** | An officially maintained corpus is insufficient to pin its own language: 28.5% three-way disagreement. | Full corpus against three implementations; no model spend. |
| **Laboratory-only** | | |
| **RQ2** | A reconstruction can perform the operations that produce the next generation, preserving identity. | One lineage; kernel acts executed by a prior reconstruction; a further generation landed the same identity. Authors' hardware. |
| **RQ4** | Witnessed failures converted into obligations reduce consequential divergence: 15 closed against 1 opened, $p = 2.6 \times 10^{-4}$. | Paired per-probe, codex family, $n = 7/n = 5$. Correlated probes; analysis specified after an initial instrument failure. |
| **RQ4** | Newly discovered failures per generation declined (3,3,3,2,1,0). | Authors' classification, not mechanically derived. |
| **Negative** | | |
| **RQ4** | The divergence reduction does not replicate in the second producer family: 12 closed, 12 opened, $p = 0.58$. | $n = 1$ per boundary. Class decomposition offers a falsifiable explanation. |
| **RQ5** | Sampled reconstruction recovers only 17.6% of the known human divergence surface, against a predicted $\geq 30\%$. | Preregistered and falsified; a second producer family added 2.6 points. |
| **RQ5** | No novel behavioral ambiguity was found (strict criterion: zero). | Below the instrument's sensitivity on a heavily contaminated target. |
| | An aggregate divergence count is unable to detect the repair effect it was first used to claim. | Preregistered control at an unchanged specification refuted the original claim; it was withdrawn. |
| **Open** | | |
| | External reproduction by an independent operator. | Protocol and signed-receipt format specified; nothing returned. |
| | Whether repair completes a specification or merely relocates its failures. | Seven successive single-defect refusals in one family are consistent with both readings. |
| | Cost relative to any alternative method. | No comparative cost study exists. |

ior and expression. Our method operates on behavior, and the legal treatment of behavioral reimplementation is jurisdiction-specific and outside our competence. We flag these because the method's attractiveness as a means of escaping lock-in makes them immediate, not because this paper resolves them.

# 9 Limitations and Threats to Validity

**Acceptance criteria are finite and incomplete.** Every obligation in this work is an executing program over finitely many inputs, and Proposition 1 applies to any part of it that compares fixed observations. A specification that admits an implementation has established conformance to itself and nothing more. Failure to detect a defect is not evidence that none exists; the `jq` arm's zero novel-ambiguity result is a statement about our instrument's sensitivity.

**Blindness is declared, not enforced.** The harness does not route the original implementation into the reconstruction workspace, and the specification states explicitly that content cannot prove blindness. Our indirect evidence is artifact dissimilarity, two unrelated producer families, and a held-out battery outside the committed identity. A producer that had memorized the target would be detected by none of these with certainty.

**Specification and probe population co-evolved.** The obligations were authored by the method's designer and revised repeatedly in response to the behavior of the two producer families used to probe them. This is the correct way to develop a boundary and a confound when measuring one. A specification authored independently of the producers that probe it would be a stronger test, and we have not run one.

**The self-hosting arm was not preregistered.** It was conducted as development. Its results are therefore measurements of a system under repeated repair rather than the outcome of a design fixed in advance, and the declining failure-per-

generation series in particular is the authors' classification of their own development history.

**External target selection.** `jq` was chosen for having multiple independent implementations and an official corpus, which makes it informative about divergence and uninformative about agreement, because it is heavily represented in producer training data [35, 6]. One target does not support claims about external software generally.

**Small samples and correlated observations.** The paired divergence analysis rests on 7 and 5 samples in one family and 1 in the other. Probes within a sample are not independent: they are drawn from the same seeded battery and a single implementation defect affects many at once. We report exact tests on the paired per-probe unit and treat the result as exploratory. The analysis was specified after an initial instrument failure, which is adaptive analysis and a real threat; the control that refuted the first attempt was itself preregistered, which partially mitigates it.

**The probe population that measured the repair also selected it.** The error-condition obligations of §7.4 were drafted from failures witnessed on the 1,395-probe battery, and the paired before-and-after comparison was then computed on that same battery. The measured contraction therefore demonstrates effective repair on the population that motivated it, and does not establish generalization to unseen behavior. A stronger design freezes the specification, discovers failures on one probe collection, constructs repairs, and evaluates fresh reconstructions against a collection never used for selection. We did not run that design, and the result should be read accordingly.

**Overfitting during repair.** Obligations drawn from observed failures may encode the particular producers' habits rather than the target's semantics. Three candidate obligations were rejected on exactly this ground after measurement contradicted them, which indicates the failure mode is live rather than hypothetical.

**Continuous integration is not external reproduction.** Our criteria are re-executed on platform configurations we do not administer, which tests portability. It runs our workflow, on our specification, triggered by us. It is not an independent operator reproducing a result.

**No external reproduction exists.** This is the limitation a reader should weigh most heavily. The protocol's own M2 leg, applied to the self-hosting results, is incomplete in the technical sense of §3: a hard condition is declared and unmeasured. A protocol and signed-receipt format for an external operator are specified in advance, and nothing has been returned. Two records in the repository named for the outside re-earn are same-host dry runs of that instrument, not external replications. Every self-hosting result here is a laboratory result.

**No baseline comparison.** We did not run differential testing, property-based testing, or synthesis tooling as alternative means of finding the defects reported here, so we make no claim that reconstruction finds them more cheaply or more completely than existing techniques would.

**Three generations is not an induction.** Nothing in the machinery references generation depth. A lineage that held three times may fail the fourth.

# 10 Conclusion

Reproducibility, as the field has developed it, preserves computational artifacts: given the same inputs, recover the same bytes. We asked a different question — whether the requirements a system satisfies are sufficient to obtain a new implementation that satisfies them too — and found it to be both answerable and productive.

The Three-Machine Test separates two claims that are ordinarily conflated: that an artifact and its verdicts survive transport, and that the disclosed requirements determine the artifact. The second is the one that matters for generated software, and it is testable. A 21-component system was reconstructed from its own committed requirements by two unrelated producer families, with measured cross-family similarity of 0.123 and 32 of 37 modules structurally distinct in every reconstruction. In one case a reconstruction then performed the kernel operations for the following generation under the same identity, and its own recursive step rebuilt and verified one component claim — a demonstration that the operational interface survives reconstruction, not that a fourth complete system was grown.

Used repeatedly, reconstruction is also an instrument for finding what a specification failed to say. It located a finite corpus that admitted its own memorization, a parser boundary that six independent implementations satisfied and none could be substituted under, and a set of error conditions in an external language that its official test suite never stated. Converting those findings into obligations measurably reduced consequential divergence in one producer family and measurably did not in the other, for a reason the class decomposition explains and a pending experiment can falsify.

The honest summary of the method's cost is that three qualifying reconstructions were the end of a sequence of roughly two dozen, that the count was reset five times by the repairs the failures earned, and that no result here has been reproduced by anyone outside our environment. The open question is therefore not whether independent reconstruction can discover underspecification — it did, repeatedly, including defects in our own implementation and in the reference implementation of an established tool — but whether the adjudication it requires can be made cheap enough to run continuously. That

is a question about cost, and it needs the measurement study this paper does not contain.

## Artifact Availability

The platform, all acceptance criteria, failure witnesses, generated implementations retained as evidence, analysis scripts, preregistration, signed revision records and ledgers are in the public repository [41], at the immutable snapshot `4c5ca38`; every figure and number here resolves at that commit. The repository also carries an evidence map that ties each quantitative claim in this paper to the artifact supporting it, and records for each whether the number was re-derived from stored data, read from a dated record, or reported by the authors without separately recoverable data. Reconstructed implementations are destroyed after measurement by design, and the retained corpus is the set used for the diversity measurements of §7.1.

Two disclosures. First, one early probe record was overwritten by a later run that reused its path; because the probes are seed-determined and the implementation source was preserved, the record was regenerated exactly, matching the figures recorded at the time (587 agreements and 808 refusals over 1,395 probes). No other number in this paper depends on a file that was overwritten. Second, no new producer runs, reconstructions or external replays were performed for this manuscript; every result is drawn from the recorded history of the project and was re-derived from stored artifacts where a script existed to do so.

## References


[1] Rajeev Alur, Rastislav Bodík, Garvit Juniwal, Milo M. K. Martin, Mukund Raghothaman, Sanjit A. Seshia, Rishabh Singh, Armando Solar-Lezama, Emina Torlak, and Abhishek Udupa. Syntax-guided synthesis. In *Formal Methods in Computer-Aided Design (FMCAD)*, pages 1–8, 2013.

[2] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, et al. Program synthesis with large language models, 2021. arXiv:2108.07732.

[3] Algirdas Avižienis. The N-version approach to fault-tolerant software. *IEEE Transactions on Software Engineering*, SE-11(12):1491–1501, 1985.

[4] Earl T. Barr, Mark Harman, Phil McMinn, Muzammil Shahbaz, and Shin Yoo. The oracle problem in software testing: A survey. *IEEE Transactions on Software Engineering*, 41(5):507–525, 2015.

[5] Juan Benet. IPFS — content addressed, versioned, P2P file system, 2014. arXiv:1407.3561.

[6] Nicholas Carlini, Daphne Ippolito, Matthew Jagielski, Katherine Lee, Florian Tramèr, and Chiyuan Zhang. Quantifying memorization across neural language models. In *International Conference on Learning Representations (ICLR)*, 2023. arXiv:2202.07646.

[7] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, et al. Evaluating large language models trained on code, 2021. arXiv:2107.03374.

[8] T. Y. Chen, S. C. Cheung, and S. M. Yiu. Metamorphic testing: A new approach for generating next test cases. Technical Report HKUST-CS98-01, The Hong Kong University of Science and Technology, 1998. arXiv:2002.12543.

[9] Elliot J. Chikofsky and James H. Cross. Reverse engineering and design recovery: A taxonomy. *IEEE Software*, 7(1):13–17, 1990.

[10] Koen Claessen and John Hughes. QuickCheck: A lightweight tool for random testing of Haskell programs. In *ICFP*, pages 268–279, 2000.

[11] Edmund Clarke, Orna Grumberg, Somesh Jha, Yuan Lu, and Helmut Veith. Counterexample-guided abstraction refinement. In *Computer Aided Verification (CAV)*, volume 1855 of *LNCS*, pages 154–169, 2000.

[12] Ludovic Courtès. Functional package management with guix. In *European Lisp Symposium*, 2013. arXiv:1305.4584.

[13] Richard A. DeMillo, Richard J. Lipton, and Frederick G. Sayward. Hints on test data selection: Help for the practicing programmer. *IEEE Computer*, 11(4):34–41, 1978.

[14] Eelco Dolstra, Merijn de Jonge, and Eelco Visser. Nix: A safe and policy-free system for software deployment. In *Proceedings of the 18th USENIX Conference on System Administration (LISA)*, pages 79–92, 2004.

[15] Dave E. Eckhardt and Larry D. Lee. A theoretical basis for the analysis of multiversion software subject to coincident errors. *IEEE Transactions on Software Engineering*, SE-11(12):1511–1517, 1985.

[16] Michael D. Ernst, Jeff H. Perkins, Philip J. Guo, Stephen McCamant, Carlos Pacheco, Matthew S. Tschantz, and Chen Xiao. The Daikon system for dynamic detection of likely invariants. *Science of Computer Programming*, 69(1–3):35–45, 2007.

[17] Benny Godlin and Ofer Strichman. Regression verification. In *Design Automation Conference (DAC)*, pages 466–471, 2009.

[18] Sumit Gulwani. Automating string processing in spreadsheets using input-output examples. In *POPL*, pages 317–330, 2011.

[19] Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, et al. Measuring coding challenge competence with APPS. In *NeurIPS Datasets and Benchmarks Track*, 2021. arXiv:2105.09938.

[20] Daniel Jackson. Alloy: A lightweight object modelling notation. *ACM Transactions on Software Engineering and Methodology*, 11(2):256–290, 2002.

[21] Yue Jia and Mark Harman. An analysis and survey of the development of mutation testing. *IEEE Transactions on Software Engineering*, 37(5):649–678, 2011.

[22] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.06770.

[23] John C. Knight and Nancy G. Leveson. An experimental evaluation of the assumption of independence in multiversion programming. *IEEE Transactions on Software Engineering*, SE-12(1):96–109, 1986.

[24] Ramana Kumar, Magnus O. Myreen, Michael Norrish, and Scott Owens. CakeML: A verified implementation of ML. In *POPL*, pages 179–191, 2014.

[25] Chris Lamb and Stefano Zacchiroli. Reproducible builds: Increasing the integrity of software supply chains. *IEEE Software*, 39(2):62–70, 2022.

[26] Xavier Leroy. Formal certification of a compiler backend, or: Programming a compiler with a proof assistant. In *POPL*, pages 42–54, 2006.

[27] Bev Littlewood and Douglas R. Miller. Conceptual modeling of coincident failures in multiversion software. *IEEE Transactions on Software Engineering*, 15(12):1596–1614, 1989.

[28] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code generation. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2305.01210.

[29] William M. McKeeman. Differential testing for software. *Digital Technical Journal*, 10(1):100–107, 1998.

[30] Ralph C. Merkle. A digital signature based on a conventional encryption function. In *Advances in Cryptology — CRYPTO '87*, volume 293 of *LNCS*, pages 369–378, 1988.

[31] Bertrand Meyer. Applying "design by contract". *IEEE Computer*, 25(10):40–51, 1992.

[32] Jan Nieuwenhuizen, Ludovic Courtès, et al. Bootstrappable builds, 2026. `https://bootstrappable.org/`, accessed October 2026.

[33] David L. Parnas and Paul C. Clements. A rational design process: How and why to fake it. *IEEE Transactions on Software Engineering*, SE-12(2):251–257, 1986.

[34] Hammond Pearce, Baleegh Ahmad, Benjamin Tan, Brendan Dolan-Gavitt, and Ramesh Karri. Asleep at the keyboard? assessing the security of GitHub Copilot's code contributions. In *IEEE Symposium on Security and Privacy (S&P)*, pages 754–768, 2022.

[35] Oscar Sainz, Jon Ander Campos, Iker García-Ferrero, Julen Etxaniz, Oier Lopez de Lacalle, and Eneko Agirre. NLP evaluation in trouble: On the need to measure LLM data contamination for each benchmark. In *Findings of EMNLP*, 2023. arXiv:2310.18018.

[36] Sergio Segura, Gordon Fraser, Ana B. Sánchez, and Antonio Ruiz-Cortés. A survey on metamorphic testing. *IEEE Transactions on Software Engineering*, 42(9):805–824, 2016.

[37] Armando Solar-Lezama, Liviu Tancau, Rastislav Bodík, Sanjit Seshia, and Vijay Saraswat. Combinatorial sketching for finite programs. In *ASPLOS*, pages 404–415, 2006.

[38] Ken Thompson. Reflections on trusting trust. *Communications of the ACM*, 27(8):761–763, 1984.

[39] David A. Wheeler. *Fully Countering Trusting Trust through Diverse Double-Compiling*. PhD thesis, George Mason University, 2009. arXiv:1004.5534.

[40] Xuejun Yang, Yang Chen, Eric Eide, and John Regehr. Finding and understanding bugs in C compilers. In *PLDI*, pages 283–294, 2011.

[41] Rafael Zamora-Resendiz. Reticuli, 2026. Repository snapshot `4c5ca38`. `https://github.com/rz4/reticuli/tree/4c5ca38`.

# A The reconstruction procedure

**Decomposition.** The self-hosting target is decomposed into 21 components ordered by dependency. Each component declares the pinned inputs its obligations read, the outputs a producer must supply, and the components it depends on. The decomposition is part of the committed material and therefore part of the identity.

**Per-component reconstruction.** For each component in dependency order, the harness materializes a workspace containing the committed content only: the acceptance programs, fixtures, pinned inputs, and the recipe. Already-reconstructed dependencies are placed in the workspace as context, because an implementation of a higher layer must import the lower layers it builds on. The original implementation of the component under reconstruction is not present in the workspace at any point.

A producer is then invoked on the workspace and its output sealed. If the component's obligations fail, the producer is re-invoked with the failure output, up to a per-component attempt limit. Attempt counts are recorded and reported raw (§C); a reconstruction that needed retries is not thereby disqualified, since the obligations are the acceptance condition and not the number of attempts.

**Whole-system judgment.** After all components are reconstructed, the assembled tree is judged against the unchanged whole-system identity with all 37 generated modules substituted for the originals. This is a stronger condition than per-component acceptance: each component passed its own obligations in isolation, and the assembled system must additionally satisfy the system-level obligations, including the recursive audit of the dependency chain.

**The blindness protocol, and its limits.** Blindness means exactly this: the harness does not place the original implementation in the producer's workspace, and the producer is invoked with no reference to it. It does not mean, and the protocol cannot establish, that the producer has no prior exposure to the target. The supporting evidence is indirect and reported in §7.1: measured artifact dissimilarity, two unrelated producer families, and a held-out battery maintained outside the committed identity. The specification states the limitation directly: producer independence is recorded, never enforced, because content cannot prove blindness.

**The recursive step.** A qualifying reconstruction must also perform the operations the system exists to perform. The step exports a claim through the reconstruction's own command-line interface, imports it into a fresh workspace, rebuilds its generated outputs through a producer, and checks that the resulting identity equals the original and that the parent verifies the child. In the generational trials, every such operation is executed by a prior reconstruction rather than by the shipped implementation.

The step operates on one component claim, by default the core layer, and not on the whole system. It establishes that a reconstruction can drive the export, rebuild and verification path. It does not establish that a further complete system was grown.

# B Verdicts sealed as claims

A verdict recorded in a log is unverifiable. Each qualifying reconstruction is therefore sealed as a claim whose own acceptance procedure re-executes the conditions that qualified it: identity under full substitution, the held-out battery, and the operational steps. A claim cannot seal unless its obligations pass, so a non-qualifying trial produces no sealed manifest. We verified this property directly during the evidence audit for this manuscript: of seven trial directories, the five qualifying ones carry a sealed manifest and the two non-qualifying ones carry none.

The construction made one catch of its own before any trial sealed: it refused a seal over a boundary whose acceptance obligations named a path outside the claim, which was then corrected.

The regress this invites terminates deliberately. Judges are auditable by the machinery they judge with, and the one operation that is never automated is authorization. Adopting an obligation, and so moving an identity, requires a human signature; every obligation discussed in this paper carries one, recorded alongside the failure that selected it.

# C Detailed measurements

## C.1 Producer cost and timing

Across 26 recorded whole-system reconstructions, each of 21 components:

| family | median wall-clock | range | per-component retries |
|---|---|---|---|
| codex (`gpt-6-sol`) | 40 min | 27–211 min | 0–1 |
| claude (`sonnet`) | 186 min | 147–438 min | 0–4 |

The declared cost envelope for a whole-system claim is `usd = 40.0`, pinned from the project's own ledger history with headroom. The codex family ran on a flat-rate plan and recorded zero metered spend against that ceiling; the claude family likewise ran on a subscription rather than metered API billing. Consequently the dollar figures in the repository understate the market cost of these reconstructions, and we make no cost claim from them (§8).

## C.2 Implementation agreement per component

Mean pairwise raw text similarity across the seven independent reconstructions (the shipped implementation excluded), by component, highest to lowest. The pattern of interest is that agreement tracks obligation density, and that every failure witness recorded during the trials occurred at a low-agreement component.

| component | agreement | note |
|---|---|---|
| `_kernel/core.py` | 0.377 | most heavily pinned component |
| `feedback.py` | 0.244 | |
| `_kernel/seal.py` | 0.238 | |
| `_kernel/attest.py` | 0.216 | |
| `_kernel/recipe.py` | 0.209 | |
| `_kernel/identity.py` | 0.197 | |
| `_util.py` | 0.182 | |
| `hooks.py` | 0.144 | |
| `transfer.py` | 0.140 | |
| `reference.py` | 0.134 | |
| `_cli/output.py` | 0.131 | |
| `pack.py` | 0.117 | witness: record round-trip |
| `_kernel/run.py` | 0.116 | witness: sandbox floor |
| `attest.py` | 0.114 | |
| `_cli/handlers.py` | 0.111 | |
| `assess.py` | 0.110 | |
| `render.py` | 0.104 | |
| `record.py` | 0.102 | |
| `authoring.py` | 0.099 | |
| `_kernel/crosscheck.py` | 0.090 | |
| `_cli/parser.py` | 0.088 | |
| `_cli/views.py` | 0.084 | |
| `reuse.py` | 0.075 | |
| `launcher.py` | 0.067 | |
| `_cli/statusview.py` | 0.066 | |
| `_kernel/build.py` | 0.064 | |
| `kernel.py` | 0.061 | |
| `registry.py` | 0.060 | witness: staging, resolution |
| `_cli/report.py` | 0.057 | |
| `heldout.py` | 0.049 | |
| `_cli/verbs.py` | 0.046 | |
| `_cli/dispatch.py` | 0.024 | least constrained component |

Package initialization files and the top-level entry point are omitted; they agree at 0.58–0.76 and contain almost nothing to write. We state the relationship between agreement and obligation density as an observation rather than a tested correlation: the components are not independent observations, and obligation density was not assigned experimentally.

## C.3 Diversity instrument

The similarity figures in §7.1 were produced by an instrument fixed before the measurements reported here were taken, and validated by reproducing an earlier reading of the project's own corpus. On the raw channel it recovers that reading to within $0.003$ on seven of eight per-reconstruction scores, with the eighth high by $0.0070$, and reproduces the count of 32 of 37 components structurally distinct in every reconstruction. It reports two channels: raw text, and text with comments, docstrings and layout normalized away via syntax-tree unparsing. The normalized channel runs 0.05 to 0.09 higher, because a substantial part of what separates two reconstructions as text is prose.

# D External arm detail

## D.1 Materials

The corpus is the official `jq` 1.8.1 test suite, 522 parseable cases, frozen with checksums. All three implementations return the same verdict on 373 of them — 332 passed by all three, 41 failed identically by all three — and differ on the remaining 149. The oracles are `jq` 1.8.1, `gojq` 0.12.19 and `jaq` 3.1.1, also frozen. The oracles adjudicate disagreement and are never used as acceptance criteria, so that no obligation is defined by running a reference implementation inside the gate.

The initial specification $S_0$ is the evaluation-core slice: the 465 cases exercising expression semantics, excluding regular expressions, dates, input/output, modules and command-line flags, and excluding approximately 43 cases that fail even for the reference implementation under a plain runner. The carve rule is "the case drives only the evaluator", applied by a tagger script and reviewed once.

## D.2 Obligation inventories

$S_1$ adds 16 computed and metamorphic laws evaluated over inputs seeded from entropy at evaluation time: indexing and length relations computed independently by the checker; composition against sequencing; idempotence of sorting; round-tripping of serialization; and concatenation length additivity. Each was validated on three fresh seeds against the reference before adoption.

$S_2$ adds 28 error-condition laws requiring that generated ill-typed inputs raise, covering wrong-typed arguments to entry conversion, transposition, joining and character conversion; malformed path arrays given to path deletion and assignment; non-path expressions given to path extraction; mismatched operand types for containment and indexing; and arithmetic on incompatible types. Each condition was first verified unanimous across all three implementations, and the battery validated on five fresh seeds. A sanity guard accompanies the battery so that an implementation refusing all inputs cannot satisfy it.

Three drafted laws were rejected before adoption because measurement contradicted them. The reference's refusals for list reversal, transposition and vacuous quantification proved value-dependent rather than type-dependent; reversing the number $0$ yields the empty list while reversing $5$ raises, because the definition routes through a length that is zero in the first case. One candidate was abandoned entirely, its refusal condition not being expressible as a property of types.

## D.3 Probe battery and per-sample results

The differential battery is 1,395 deterministic seeded probes over program and input pairs that the corpus does not contain, executed against four implementations. The human-divergence surface is the 343 probes on which `gojq` or `jaq` disagrees with the reference, giving a base rate of 24.4%; it was frozen independently of any reconstruction.

| boundary | sample | divergent probes | on human surface |
|---|---|---|---|
| $S_1$ | codex 1 | 79 | 47 (59%) |
| $S_1$ | codex 2 | 52 | 43 (83%) |
| $S_1$ | codex 3 | 66 | 44 (67%) |
| $S_1$ | pooled (distinct) | 92 | 51 (55%) |
| $S_1$ | claude 1 | 34 | 26 (76%) |

Of the 92 pooled codex divergences, 43 were flagged by all three samples, 19 by two and 30 by one, which indicates the battery measures structure rather than noise. Of the 43 flagged by all three, 40 (93%) lie on the human-divergence surface.

The claude sample is 3,263 lines against the codex samples' 869, a ratio of 3.8, which independently reproduces the family size signature measured on the self-hosting corpus. It is also qualitatively different: 17 of its 34 divergences are cases where both implementations succeed with different values, where the codex samples' divergences were dominated by excess permissiveness.

## D.4 Divergence classes by family

The class decomposition that explains the negative cross-family result in §7.4:

| family | boundary | permissive | value split | sample error |
|---|---|---|---|---|
| codex | $S_1$ | 35.3 | 19.3 | 10.0 |
| codex | $S_2$ | 27.5 | 13.0 | 8.0 |
| claude | $S_1$ | 6 | 17 | 11 |
| claude | $S_2$ | 5 | 20 | 9 |

Excess permissiveness is 54% of the codex family's divergence and 18% of the claude family's. The $S_2$ repair targeted permissiveness, and permissiveness fell in both families. The aggregate moved only for the family in which that class dominates.

### D.5 The unresolved election

Of the programs diverging consistently at both boundaries, 19 reduce to four classes: six differing only in error message text, which project precedent leaves unpinned; nine with a single reimplementation as the outlier, admitting a mechanical consensus resolution; three constituting a genuine election, where indexing an object by a string key returns null in the reference and raises in both reimplementations; and one liveness divergence recorded as a finding rather than pinned. The three-case election is unresolved. Electing the reference requires members to replicate a permissive edge two independent implementations treat as an error; electing the majority produces a specification that refuses the artifact it was recovered from.

## E Reproduction

The repository contains the protocol for an independent operator to re-earn a sealed trial, together with the signed-receipt format and a continuous-integration workflow that checks a returned receipt against the bundle's own public key. Re-earning a sealed trial claim requires the checkout, the retained reconstruction lineage, and one audit command; it executes the acceptance obligations and does not require a producer or any model spend.

Reproducing a reconstruction from scratch requires a producer. The expected cost is given in §C; because both families were run on flat-rate plans, an operator billed per token should expect a materially higher figure, and the declared envelope of `usd = 40.0` per whole-system claim is the project's own stated ceiling rather than a measured market price.

No external operator has returned a receipt. The two records in the repository named for this condition are same-host dry runs of the instrument, retained to show the measurement path works, and each still carries a placeholder note in place of an operator's signature.